Codebase walkthrough

projectAlp

DTU IQAC data portal + APAR (Annual Performance Appraisal Report) system
Folder structure · file-by-file purpose · how data flows end to end

Generated 12 Oct 2026 from the repository on branch `final` (commit 5dabd55). Everything below was read from the source; file names and line counts refer to the repo as it stands today.

---

1. What the project is, and the stack
2. System architecture (containers and services)
3. Folder structure
4. Backend: what each file is for
5. Frontend: what each file is for
6. Database: collections and how they relate
7. Roles and permissions
8. Data flow, scenario by scenario
9. Root-level and miscellaneous files
10. Things worth knowing (quirks and cleanup candidates)

1. What the project is

projectAlp is one codebase with **two portals** that share a backend, a database and a user table:

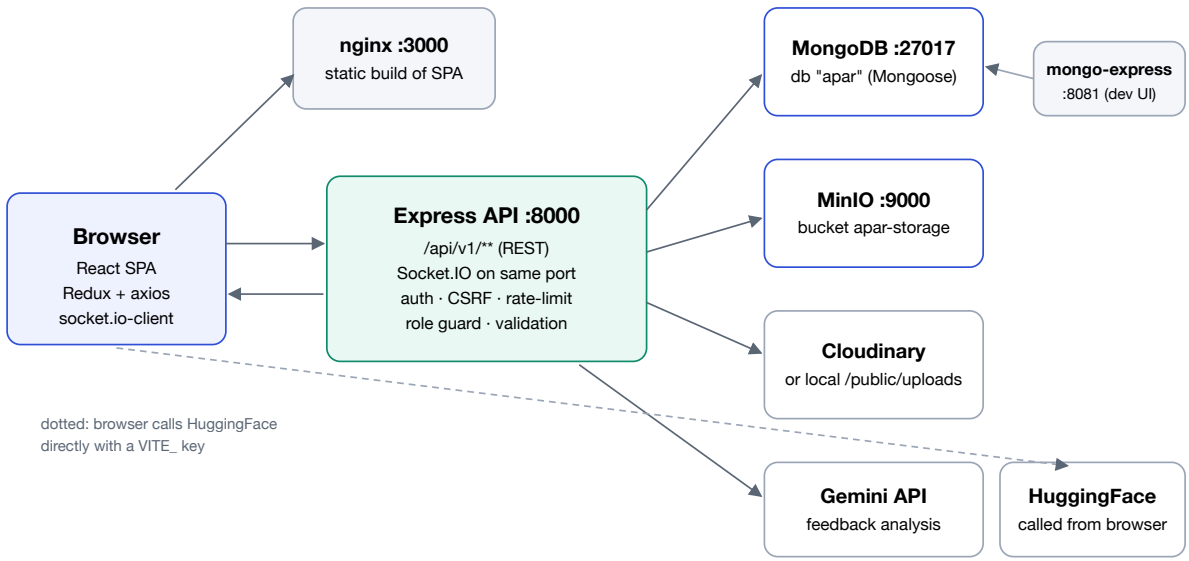
Portal	URL prefix (frontend)	Purpose
IQAC portal	`/login`, `/app/*`	Internal Quality Assurance Cell. IQAC Head and Department HODs enter and review institutional data (publications, patents, students, training, MoUs, finance, etc. — **37 resource types**), see dashboards and export reports, and run AI feedback analysis.
APAR portal	`/apar/login`, `/apar-form`, `/apar/*`	Annual Performance Appraisal Report. A faculty member fills a 6-part form; a Reporting Officer assesses it; a Reviewing Officer remarks on it; the Dean monitors status. Research data from the IQAC side is **auto-synced into the faculty member's draft form**.

The landing page `/` (`Landing.jsx`) lets the user pick a portal.

Tech stack

Layer	Technology
Frontend	React 19, Vite 7, React Router 7, Redux Toolkit, Tailwind CSS 4, axios, socket.io-client, recharts (charts), jsPDF / docx / xlsx / file-saver (exports), react-markdown (AI output), sonner + react-toastify (toasts), dnd-kit.
Backend	Node 20, Express 5 (ES modules), Mongoose 9 on MongoDB 7, Socket.IO 4, JWT (jsonwebtoken) in HttpOnly cookie, bcrypt, Zod validation, helmet, express-rate-limit, csrf-csrf (double-submit CSRF), multer (uploads), minio client, cloudinary, @google/generative-ai (Gemini), xlsx.
Storage	MongoDB (database name `apar`), MinIO (S3-compatible, APAR documents and profile files), Cloudinary or local `public/uploads` (IQAC attachments).
Deploy	Docker Compose locally (frontend served by nginx on :3000, backend on :8000); Railway guide in `deploy.md`; `vercel.json` in the frontend for SPA rewrites.

2. System architecture



In `docker-compose.yml` there are six services: `frontend` (nginx), `backend`, `mongodb` (volume `mongo-data`), `mongo-express`, `minio` (volume `minio-data`) and `minio-init` (one-shot job that creates the `apar-storage` bucket). The backend only starts once MongoDB is healthy and the bucket exists. The backend also calls `ensureAparBucket()` on boot as a safety net.

3. Folder structure

```
projectAlp/
├── backend/                    Node/Express API  (195 tracked files under src/)
│   ├── src/
│   │   ├── index.js            Entry: HTTP server + Socket.IO + Mongo connect + MinIO bucket check
│   │   ├── app.js              Express app: middleware chain and ALL route mounting
│   │   ├── db/index.js         Mongoose connect (MONGODB_URI + "/apar")
│   │   ├── config/             Roles, route-permission table, Socket.IO server, IQAC approval registry
│   │   ├── middlewares/        auth, CSRF, rate-limit, role checks, multer, Zod validate, error handler
│   │   ├── routes/             One router per resource (47 files)  → URL → controller
│   │   ├── controllers/        HTTP layer: read req, validate, call data-access, send ApiResponse
│   │   ├── data-access/        DB layer: all Mongoose queries, merge/dedupe logic, triggers APAR auto-sync
│   │   ├── models/             Mongoose schemas (20 models, 20 collections incl. AparForm)
│   │   ├── validators/         Zod schemas for auth, APAR form, APAR profile
│   │   ├── services/           MinIO client, temp-document store for APAR PDFs
│   │   ├── utils/              JWT, passwords, ApiError/ApiResponse, Cloudinary, APAR access and auto-sync, notifications
│   │   └── scripts/seedUser.js Seeds a first user (used by `docker compose exec backend node src/scripts/seedUser.js`)
│   ├── scripts/                Ops scripts: seed-user, reset/clear password flag, de-dupe APAR forms, API smoke test
│   ├── tools/                  route_tester.js (hits every route), Gemini key checks
│   ├── public/                 Static files: temp/ (multer landing zone), uploads/ (local fallback storage)
│   ├── *.js (root)             One-off debugging scripts (check_apar_*.js, test_db_*.js, inspect_db.js ...)
│   ├── dummy_*.json            Sample data;  sample_form.json = an exported APAR form
│   ├── Dockerfile, .env.example, package.json
│
├── frontend/                   React SPA
│   ├── index.html, vite.config.js, nginx.conf, vercel.json, Dockerfile, .env.example
│   ├── public/                 Logos (DTU)
│   ├── scripts/tester.js       Form-submission tester + sample logs
│   └── src/
│       ├── main.jsx            Entry: Redux Provider, SocketProvider, router table
│       ├── App.jsx             IQAC shell layout (Sidebar + Header + Outlet)
│       ├── api/Api.js          axios wrapper: cookies, CSRF header, role header, 401 redirect
│       ├── store/              Redux store + slices: auth (IQAC) and aparAuth (APAR)
│       ├── services/           ~45 thin API clients, one per backend resource (+ index.js barrel)
│       ├── config/             tableConfig.js (the resource catalogue), role permissions, duplicate rules, AI prompts
│       ├── pages/              Route-level screens (Landing, Auth/Login, AparForm, IqacReports, Apar/*)
│       ├── components/         Shared UI: tables, dynamic forms, sidebar/header, route guards, modals, bell
│       ├── context/            SocketContext (one global socket), IqacFilterContext (year/dept filter)
│       ├── hooks/              useAparRealTimeSync, useNotifications
│       ├── utils/              CSRF token cache, academic-year helpers, validators, feedback-file parser
│       └── constants/roles.js
│
├── docker-compose.yml          frontend + backend + mongodb + mongo-express + minio + minio-init
├── README.md  setup.md  deploy.md  SECURITY_AUDIT_REPORT.md  apar_presentation_content.md
├── requirements.txt            Flat list of npm packages (documentation only)
├── rewrite_iqac_exports.js     One-time codemod that rewrote export code in IqacReports.jsx
├── diff.txt                    Stray UTF-16 git diff of tableConfig.js
└── .windsurf/workflows/        An editor workflow note (cgpa.md)
```

4. Backend: what each file is for

4.1 Entry and wiring

File	Purpose
`src/index.js`	Loads env, creates the HTTP server from the Express app, attaches Socket.IO (`initializeSocket`), connects MongoDB, ensures the MinIO bucket exists, listens on `PORT` (default 8000).
`src/app.js`	Builds the middleware pipeline (see section 8.1) and mounts every router under `/api/v1/...` . A helper `registerProtectedRoute(path, router)` mounts a router behind `authenticate` + a per-path role guard. Also contains a global middleware that normalises academic-year strings (`2026-2027` → `2026-27`) in every request body and query.
`src/db/index.js`	`connectDB()` : appends `/apar` to `MONGODB_URI` (fallback `mongodb://localhost:27017/apar`); exits the process on failure.

4.2 config/

File	Purpose
`roles.js`	The four **system roles**: IQAC Head, Dean, Department HOD, Faculty Member; plus `normalizeRoleValue()` that maps aliases (`iqac_head` , `faculty` ...) to canonical names.
`aparRoles.js`	The four **APAR roles**: Officer (Graded), Reviewing Officer, Reporting Officer, Dean; same normaliser pattern.
`permissions.js`	The route-permission table. Builds `routePermissions[path][METHOD] = [roles]` for the 38 protected resource paths. Used by `createRouteGuard` . Rules in section 7.
`socket.js`	Socket.IO server. Optional JWT check on handshake, room management (`join_apar_room` , `join_iqac_admin_room` , `join_notifications`), and emit helpers: `emitNewEntry` , `emitUpdateEntry` , `emitDeleteEntry` , `emitBulkEntries` , `emitAparDataUpdated` , `emitCrossFacultyUpdate` , `emitAparUpdateToIqac` , `emitNotification` . Resolves Mongo ObjectIds to readable `user_id` / `faculty_id` so rooms match.
`iqacApprovalRegistry.js`	Maps a resource id (e.g. `journal_research_papers`) to its title, key field and the `set()` function of its data-access module. Lets the approval flow create the final record once all faculty approve.

4.3 middlewares/

File	Purpose
`auth.middleware.js`	`authenticate` : reads the JWT from the `access_token` cookie (or `Authorization: Bearer`), verifies it, loads the user, and requires the token to equal the user's stored `session_token` and be unexpired. Sets `req.user` (id, userId, role = active role from token, systemRole, aparRole, departmentId, academicYear). `createRouteGuard(path)` applies the permissions table, falling back to the user's system role.
`csrf.middleware.js`	Double-submit CSRF (`csrf-csrf`). `GET /api/v1/csrf-token` issues a token; all non-GET/HEAD/OPTIONS requests must send it in `x-csrf-token` .
`rate-limit.middleware.js`	Global limit 300 req / 15 min per IP; login limit 20 / 15 min (both configurable by env).
`iqac-role.middleware.js`	`requireIqacRole` : only IQAC Head or Department HOD (active or system role). Guards `/dashboard` and `/feedback` .
`validate.middleware.js`	Runs a Zod schema against `{body, params, query}` .
`multer.middleware.js`	Disk upload into `public/temp` , 10 MB limit, allows images/PDF/Office files.
`apar-document-upload.middleware.js`	Multer for APAR PDFs, saved under `storage/temp/apar-documents` with a UUID name.
`error.middleware.js`	Final error handler: turns `ApiError` / Mongoose / Multer errors into the standard JSON error shape.
`must-change-password.middleware.js`	Blocks everything except logout/profile/change-password when `mustChangePassword` is set. Defined but not wired into `app.js` .

`parseFormData.middleware.js`	Empty file.

4.4 routes/ → controllers/ → data-access/ → models/

There are four kinds of routes.

Group	Mount path	Files and behaviour
Auth	`/api/v1/auth` `/api/v1/apar/auth`	`auth.routes.js` + `auth.controller.js` (IQAC login, logout, profile, verify-role, change-password, register, allowed-roles, user-ids). `apar.auth.routes.js` + `aparAuth.controller.js` (same for APAR, plus avatar upload to MinIO; token carries the chosen academic year).
37 IQAC resources	`/api/v1/<resource>` e.g. `/journal_research_papers`	Each has `routes/<r>.routes.js`, `controllers/<r>.controller.js`, `data-access/<r>.data-access.js`. Standard endpoints: `POST /set`, `GET /get`, `GET /get/:id`, `PUT /update/:id`, `DELETE /delete/:id`. Mounted with `authenticate` + role guard.
APAR core	`/api/v1/apar/mongo` `/api/v1/apar/profile`	`apar.mongo.routes.js` + `apar.mongo.controller.js` (2,576 lines, the heart of APAR: get/save/submit/delete form, save-to-monthly, reporting and reviewing queues, dean status, history, IQAC sync, cross-faculty de-dupe). `apar-document.controller.js` (temp and direct PDF upload, stream, delete). `duplicate-check.controller.js`. `apar.profile.routes.js` + `apar.profile.controller.js` (faculty profile CRUD).
Cross-cutting	`/upload`, `/notifications`, `/iqac-approvals`, `/dashboard`, `/feedback`, `/auth/admin/users`	`upload.controller.js` (generic file upload), `notification.controller.js` (create + list/read/delete + role broadcast), `iqacApproval.controller.js` (faculty-approval workflow), `dashboard.controller.js` (752 lines of aggregations), `feedback.controller.js` (Gemini analysis of uploaded Excel/CSV), `user_management.controller.js` (admin CRUD of users).

The 37 IQAC resources, and which collection stores each

Many resources share one collection and are told apart by a `type` field. That is why 37 resources fit in far fewer collections.

Model (collection)	`type` values	Resources stored there
Publication (`publications`)	journal, conference, book	journal_research_papers, conference_research_papers, books_chapters_published
Patent (`patents`)	—	patents
ResearchProject (`research_projects`)	funding, consultancy, corporate_training	research_funding, revenue_from_consultancy, revenue_from_corporate_training
PhdDefence (`phd_defences`)	—	phd_defence
FacultyActivity (`faculty_activities`)	fdp, visit, affiliation, ict, award, econtent, financial_support	faculty_development_programs, faculty_visits, professional_affiliations, teachers_using_ict, research_innovation_awards, e_content_developed, financial_support_events
Collaboration (`collaborations`)	activity, exchange, mou, outreach	collaborative_activities, collaborative_research_exchange, functional_mous, extension_outreach_activities
StudentActivity (`student_activities`)	exam, higher_ed, performance, financial_support	students_competitive_exams, students_higher_education, student_performance_activities, student_financial_support_events
Training (`trainings`)	staff, professional, mentoring, capability	staff_training, professional_training_staff, mentors_stress_support, capability_enhancement_schemes
DepartmentResource (`department_resources`)	library, it_stock, scheme	dept_library_books, it_infrastructure_stock, dept_professional_schemes
Teaching (`teaching_methods`)	teaching_method	student_centric_methods

Programme (`programmes`)	standard, field_research	programmes, programmes_with_field_research
Own collection each	—	faculty, students, courses, departments

4.5 data-access/ (the DB layer)

One file per resource plus `users.data-access.js`. This is where the real logic lives. Using `journal_research_papers.data-access.js` as the template:

- `set()` parses and de-duplicates the faculty/student/external-author arrays, adds the logged-in faculty member automatically, then looks for an existing paper with the same title + journal + year (case-insensitive). If found it **merges** the new authors into that record and appends a `change_log` entry; otherwise it creates a new one with a UUID key.
- Every write ends with `triggerAparAutoSyncMultiple(facultyIds, academicYear, entry)`, a fire-and-forget call that pushes the change into those faculty members' APAR drafts.
- `transformPaper()` reshapes the stored document into the field names the frontend table config expects; also returns `last_modified_by` and `change_count` for the audit modal.
- `get / getById / update / deletePaper` are straightforward; ids are matched against `paper_id`, `publication_id` or `_id`.

`users.data-access.js` maps the snake_case Mongo user to a camelCase object (`userId`, `aparRole`, `sessionToken` …) and handles sessions, lockouts and password updates.

4.6 models/ (selected)

File	Purpose
`user.model.js`	Login identity. `user_id` (the link to `faculty_id`), email, `password_hash`, `role`, `apar_role`, `department_id`, `reporting_officer_id`, `reviewing_officer_id`, and session fields (`session_token`, `session_expires_at`, `failed_login_attempts`, `locked_until`, `must_change_password`).
`faculty.model.js`	Faculty profile used by IQAC: `faculty_id`, name, designation, department, qualifications. Authentication data lives in `User`, not here.
`facultyProfile.model.js`	The richer self-service profile on the APAR side (basic info, qualifications with certificate URLs, etc.).
`aparForm.model.js`	One document per (`faculty_id`, `ay`) — unique index. Sections: `personal` (Part I), `teaching` (Part II), `research` (Part III: arrays for journals, conferences, books, projects, patents, awards, ...), `corporate` (Part IV), `assessment` (Part V, filled by Reporting Officer), `remarks` (Part VI, Reviewing Officer), plus `status`, `timeline`, `history[]`, `monthly_saved_at`, officer ids and query comments.
`iqacApproval.model.js`	A pending request that a set of faculty must approve before an IQAC entry is created: `resource_id`, `payload`, `approvals[]` (per faculty status), overall `status`, `created_by`.
`notification.model.js`	Persistent notifications: recipient, type (APAR_SUBMISSION, IQAC_UPDATE, IQAC_APPROVAL_REQUEST, ...), title, message, link, `isRead`.
`index.js`	Barrel that re-exports every model.
other models	Each IQAC model carries a `metadata` block: `created_by`, timestamps and a `change_log[]` of who changed what.

4.7 validators/, services/, utils/

File	Purpose
`validators/apar-auth.validator.js`	Zod schemas for login (IQAC and APAR), change-password, verify-role, allowed-roles.
`validators/apar-form.validator.js`	Schemas for draft save, full submit, reporting assessment and reviewing remarks.
`validators/apar-profile.validator.js`	Schema for profile upsert.
`validators/module.validators.js`	Shared module-level schemas.
`services/minio.service.js`	MinIO client; `ensureAparBucket`, `uploadLocalFile`, `getObject`, `deleteObject` and path builders (`profilepicture/…`, `document/<name>/<ay>/…`, `additional documents/…`, `profile/…`).
`services/apar-temp-document.service.js`	Local staging area for PDFs the faculty uploads before the form is saved: ownership sidecar JSON, pending/complete states, cleanup.

`utils/jwt.js`	`signAccessToken` / `verifyAccessToken` and the cookie name (`access_token`).
`utils/password.js`, `password-policy.js`	bcrypt hash/verify; minimum-length and complexity policy.
`utils/api-error.js`, `api-response.js`, `async-handler.js`	The response contract: `{statusCode, data, message, success}`; `ApiError` carries an HTTP status; `asyncHandler` forwards rejected promises to the error middleware.
`utils/apar-auto-sync.js`	**IQAC → APAR bridge.** Finds the faculty member's draft forms, calls `syncIqacToAparForm`, saves, creates a notification and emits a socket event.
`utils/apar-access.js`	Who may see which faculty member's APAR (self, assigned reporting/reviewing officer, IQAC/Dean) and how the "list forms" query is scoped by role.
`utils/apar-helpers.js`	`resolveToReadableId`: Mongo ObjectId → `user_id` / `faculty_id`.
`utils/notification-helpers.js`	Turns an entry into a human-readable type/summary for notifications.
`utils/cloudinary.js`	Upload/delete wrapper; deletes the temp file afterwards.
`utils/qualification.util.js`, `db-validation.util.js`, `validation.js`	Small helpers.

4.8 Operational scripts

File	Purpose
`src/scripts/seedUser.js`, `scripts/seed-user.js`	Create a user from CLI args or env.
`scripts/clear-must-change-password.js`, `scripts/reset-password.mjs` (untracked)	Fix-up utilities for accounts.
`scripts/cleanup-apar-duplicates.js`	Removes duplicate APAR forms.
`scripts/verify_api.js`, `tools/route_tester.js`	Smoke-test the API; the tester scrapes `app.js` and hits each endpoint, writing `route-test-report.json/txt`.
`tools/test_gemini.js`, `verify_key.js`	Check the Gemini key.

5. Frontend: what each file is for

5.1 Bootstrapping and state

File	Purpose
`src/main.jsx`	Router table. Public: `/`, `/login`, `/apar/login`. APAR (wrapped in `ProtectedAparRoute`): `/apar-form`, `/apar/dashboard`, `/apar/reporting`, `/apar/dean`, `/apar/profile`, `/apar/iqac-approvals`. IQAC (`ProtectedRoute + IqacRoleRoute`): `/app` with children `tables/:resourceId`, `add/:resourceId`, `reports`, `user-management`; and `/feedback-analysis`. Wraps everything in Redux `Provider` and `SocketProvider`.
`src/App.jsx`	The IQAC shell: `RoleGuard`, toaster, `IqacFilterProvider`, `Sidebar`, `Header`, `<Outlet/>`.
`src/store/index.js`	Redux store with two independent slices, `auth` and `aparAuth`. On load, it checks the URL: if it starts with `/apar` it restores the APAR session, otherwise the IQAC session. Also registers the "role provider" that `Api.js` uses to add the `X-Client-Role` header.
`src/store/slices/authSlice.js`, `aparAuthSlice.js`	Login / logout / initialise-session / verify-role thunks; hold `user`, `baseRole`, `role`, `allowedRoles`; the APAR slice also holds the chosen academic year.
`src/api/Api.js`	The one HTTP client. `withCredentials` so the cookie is sent; auto-fetches and attaches the CSRF token on POST/PUT/PATCH/DELETE (retries once on a CSRF error); unwraps the `{statusCode,data}` envelope so callers get `data` directly; on 401 redirects to `/login` or `/apar/login`. Base URL is `VITE_BASEURL`.
`src/utils/csrf.js`	Caches the CSRF token in memory.
`src/context/SocketContext.jsx`	Opens one Socket.IO connection once a user is logged in; joins the personal notification room; exposes `{socket, connected}`.
`src/context/IqacFilterContext.jsx`	The academic-year and department filter shared by the IQAC dashboard, tables and reports.

5.2 config/ — the part that drives the IQAC UI

File	Purpose
`tableConfig.js` (2,141 lines)	**The resource catalogue.** 37 entries, each declaring: `id`, title, icon, category (13 categories), `keyAccessor`, the service and function names for fetch / add / update / delete / get-by-id, how to submit (`json` or form-data), and the `columns[]` (header, accessor, type, options, required). Tables and forms are *generated* from this, which is why there is no per-resource page. Exports `resources` and `resourceMap`.
`rolePermissions.js`, `constants/roles.js`	Client-side role list and `canAccessForm/canAccessTable` (IQAC Head all; HOD all except departments, programmes, students, courses; Faculty none). The server re-checks all of this.
`aparRoles.js`	Client copy of the four APAR roles.
`duplicateDetectionConfig.js` (+ `_generated`)	Per-resource rules for which fields identify a duplicate.
`feedbackContexts.js`	System/user prompts for the nine kinds of feedback analysis.

5.3 services/ — API clients

Around 45 small classes, each `new Api(VITE_BASEURL)` with methods that map one-to-one onto backend routes. Example: `journal_research_papers.services.js` → `GET /journal_research_papers/get`, `POST /set`, `PUT /update/:id`, `DELETE /delete/:id`. `services/index.js` re-exports them all so the generic table/form code can look a service up by name (`Services[serviceName]`).

File	Purpose
`auth.service.js`, `aparAuth.service.js`	Login, logout, profile, verify-role, change-password for each portal.
`apar_form_graded.services.js`	Officer (faculty) side of APAR: `/apar/mongo/save`, `/submit`, `/save-to-monthly`, `/form`, `/list`, `/history`, document upload/delete.

`apar_form_reporting.service.js`	Reporting / Reviewing officer side: pending queues and submit assessment / remarks.
`faculty_profile.service.js`	`/apar/profile/*`.
`iqacApproval.service.js`	Create / list / decide approval requests.
`iqac-duplicate.service.js`	Calls `/apar/mongo/check-duplicate`.
`dashboard.service.js`, `notification.service.js`, `user_management.service.js`, `feedback.service.js`	The matching backend endpoints.
`huggingface.service.js`	Calls the Hugging Face router **directly from the browser** with a model fallback list, using `VITE_HUGGINGFACE_API_KEY`.

5.4 pages/

File	Purpose
`Landing.jsx`	Portal chooser.
`Auth/Login.jsx`	IQAC login (email, password, role).
`IqacReports.jsx`	Pick a period and department, pull every resource and export as PDF (jsPDF + autotable) or Excel (xlsx).
`FeedbackAnalysis.jsx`	Upload an Excel/CSV of survey responses, summarise it in the browser (`parseFeedbackFile.js`), send to an LLM, render the Markdown result.
`UserManagement.jsx`	IQAC Head creates / edits / deletes users and assigns roles, departments and officers.
`AparForm.jsx` (2,712 lines)	The APAR workspace. Hosts the six steps, loads and saves the form, switches mode by role (faculty edits; reporting/reviewing officers see a read-only form plus their section), joins the socket room for live updates, and exports the form to PDF/DOCX.
`Apar/PartIPersonal.jsx`	Part I — personal data.
`Apar/PartII.jsx`	Part II — self appraisal and teaching, with PDF proof uploads.
`Apar/PartIII.jsx`	Part III — research; uses `DynamicTableSection` for each research table and has the “Save as Monthly” button.
`Apar/PartIV.jsx`	Part IV — corporate life.
`Apar/PartV.jsx`	Part V — numerical assessment by Reporting Officer.
`Apar/PartVIRemarks.jsx`	Part VI — remarks by Reviewing Officer.
`Apar/OfficerDashboard.jsx`	Faculty landing: own forms, status, history.
`Apar/ReportingDashboard.jsx`	Reporting / Reviewing officer queue.
`Apar/DeanDashboard.jsx`	Dean’s overview of who has filled, submitted, been verified.
`Apar/Profile.jsx`	Faculty profile (personal, qualifications, documents, avatar).
`Apar/IqacApprovals.jsx`	Faculty inbox: approve / reject / correct IQAC entries that name them.
`ReportingForm.jsx`	Standalone officer assessment form.

5.5 components/

File	Purpose
`ProtectedRoute`, `ProtectedAparRoute`, `IqacRoleRoute`, `RoleGuard`, `RoleProtected`, `AccessDenied`, `NotFound`	Route guards and the “no access” screens. `RoleGuard` re-verifies the selected role with the server whenever it changes.
`Sidebar`, `Header`, `ProfileDropdown`, `AparShellHeader`, `FormPageHeader`, `LoginLayout`	Shell chrome. The sidebar lists tables and add-forms grouped by `category` from `tableConfig` and filters by role.

`Dashboard.jsx` (1,480 lines)	IQAC home: KPI cards, recharts charts, APAR status, pending approvals; data from `/dashboard/stats`.
`DynamicTablePage` → `TablePage`	Generic list page for any resource: search, filters, pagination, edit, delete, audit-log modal.
`AddPage` → `DynamicForm`	Generic create/edit page. Builds the form from the resource's columns, checks for duplicates, and decides between direct save and the approval flow.
`DynamicTableSection`	Inline editable table used inside APAR Parts II and III.
`SearchableSelect`, `FacultyMultiSelect`, `FileUpload`, `TableSkeleton`	Reusable inputs and loading states.
`DuplicateVerificationModal`, `ChangeHistoryModal`, `AuditLogModal`	Duplicate side-by-side compare; change history from `metadata.change_log`.
`NotificationBell` + `hooks/useNotifications.js`	Loads notifications over REST, appends new ones live from the socket.
`AparLogin`, `AparTimeline`	APAR login form (with role and academic-year pickers); status timeline.
`hooks/useAparRealTimeSync.js`	Joins the APAR socket room, toasts on `new_entry`, `cross_faculty_update`, `bulk_entries`, and merges the entry into the form (`mergeNewEntry`).

6. Database: how the collections relate

MongoDB has no foreign keys. Records are linked by **string ids**, and the key one is the **faculty id**, which is the same string in `users.user_id`, `faculty.faculty_id`, `aparforms.faculty_id`, and inside every IQAC record's `faculty_members[]`.

```
users.user_id  ═══  faculty.faculty_id  ═══  faculty_profiles.faculty_id
      │                      │
      │ reporting_officer_id / reviewing_officer_id  (→ other users.user_id)
      │
      ▼
aparforms { faculty_id, ay }  (unique pair)
      │  .research.journals[] / conferences[] / books[] / projects[] / patents[] / awards[] ...
      │        ▲ copies of ▲
      │        │ (auto-sync, matched by id field: paper_id, publication_id, patent_id, ...)
      │
publications · patents · research_projects · faculty_activities · collaborations · phd_defences
      │  each holds faculty_members: [{faculty_id}], department_id, academic_year, metadata.change_log[]
      │
departments.department_id ◄── department_id on users, faculty, students, courses, programmes, every IQAC record

iqac_approval_requests { resource_id, payload, approvals[{faculty_id,status}] }  → on full approval calls the
                                                                                    resource's data-access set()
notifications { recipient (user_id), type, link, isRead }
```

Important: IQAC data and APAR research data are **two copies**. IQAC collections are the institutional record; `aparforms.research.*` is the faculty member's draft copy. The sync functions keep them aligned in both directions (section 8.3 and 8.4).

7. Roles and permissions

A user has a **system role** (`users.role`) and optionally an **APAR role** (`users.apar_role`). When logging in, the user *chooses* which role to act as; that choice is written into the JWT (`role`), and `systemRole` is kept as a fallback.

System role	Portal	What it can do
IQAC Head	IQAC (+ APAR)	Everything: all resources, DELETE, departments / programmes, user management, dashboard, feedback analysis, create approval requests.
Department HOD	IQAC (+ APAR)	GET and POST/PUT/PATCH on resource routes; list endpoints such as journal papers are scoped to the HOD's own department (`getByDepartmentId`); no DELETE; no departments/programmes/students/courses admin; dashboard and feedback.
Faculty Member	APAR	POST/PUT on the 24 faculty-contributed resources (publications, patents, awards, ...); GET on faculty, students, courses, departments. No IQAC web portal access.
Dean	APAR	Dean dashboard; can read any faculty's APAR.

APAR role	Meaning
Officer (Graded)	The faculty member being appraised. Every non-Dean user can log in as this.
Reporting Officer	Fills Part V (assessment) for the officers assigned to them; may forward or raise a query.
Reviewing Officer	Fills Part VI (remarks); may accept or raise a query.
Dean	Monitoring role; also the fallback Reviewing Officer if none exists.

> **How access is enforced (defence in depth):** (1) the React route guards and `canAccessTable/Form` hide screens; (2) `authenticate` rejects missing or stale sessions; (3) `createRouteGuard` checks the method/path table in `permissions.js`; (4) `requireIqacRole` on dashboard and feedback; (5) `assertFacultyAccess` inside APAR controllers (self, assigned officer, or IQAC/Dean only).

8. How data flows

8.1 Every request: the pipeline



The response travels back as `{statusCode, data, message, success}`. `Api.js` unwraps it, so React code works with `data` only. Errors thrown as `ApiError` are formatted by `error.middleware.js`.

8.2 Login and session

1. User opens `/login` (IQAC) or `/apar/login` (APAR) and picks an email, password and **role** (APAR login also picks the academic year).
2. `Api.js` first fetches `GET /csrf-token`, then `POST /auth/login` or `/apar/auth/login` (limited to 20 attempts / 15 min).
3. The controller loads the user by email, rejects if locked (5 failed attempts → 15 min lock), checks the bcrypt hash, and checks the requested role is one the user may take (`canLoginAsRole`).
4. It signs a JWT `{sub, role, academicYear, departmentId}`, **stores the same token in `users.session_token`** with a 12-hour expiry, and sets it as an **HttpOnly cookie** `access_token` (SameSite Lax in dev, None+Secure in production).
5. The Redux slice stores the returned user and role. The socket context sees a user and connects.
6. On every later request, `authenticate` verifies the JWT *and* that it equals the token stored on the user. So **logging in again, or logging out, invalidates the old session everywhere**. A single user cannot hold two active sessions.
7. On a page reload, `initializeSession` calls `/auth/profile` (or the APAR one) to restore state from the cookie.

8.3 IQAC entry → faculty's APAR draft (auto-sync)

This is the main cross-system flow.

1. An IQAC Head/HOD opens `/app/add/journal_research_papers`. `AddPage` reads the column definitions from `tableConfig.js` and renders `DynamicForm`.
2. On submit, a duplicate check runs (`POST /apar/mongo/check-duplicate`); a match opens `DuplicateVerificationModal`.
3. **Decision point.** If the user is IQAC Head/HOD, this is a new entry (not an edit) and the data names faculty members, the page does **not** save directly: it creates an approval request (flow 8.5). Otherwise it calls the resource service → `POST /journal_research_papers/set` (multipart if a PDF is attached).
4. Controller validates required fields, uploads the PDF to Cloudinary (`link_to_paper`), and calls `set()` in data-access.
5. `set()` merges with an existing identical paper or creates a new `Publication` (type `journal`) and writes a `change_log` entry.
6. `triggerAparAutoSyncMultiple(facultyIds, ay, entry)` runs in the background. For each listed faculty member it finds forms with status `Draft` / `Query Raised`, calls `syncIqacToAparForm` (pulls matching IQAC records for that academic year into `form.research.*`) and saves.
7. If the form changed: create a `Notification` ("New IQAC Entry Synced"), push it to the user's bell via `emitNotification`, and emit `new_entry` / `update_entry` / `delete_entry` to the room `apar-<facultyId>-<ay>`.
8. If the faculty member has the form open, `useAparRealTimeSync` receives the event, toasts it and merges the entry into the form without a reload. If not, the next `GET /apar/mongo/form` runs the sync again on load.



8.4 Faculty edits APAR → IQAC (reverse direction)

- **Save draft** (`POST /apar/mongo/save`, Zod-validated): empty strings are cleaned, duplicates inside the draft are rejected, and documents are resolved (below). **If the faculty member removed a research row that had an id, the matching IQAC record is deleted** from `publications`, `patents`, etc. ("propagate faculty deletions").

- **Save as Monthly** (`POST /apar/mongo/save-to-monthly`, Part III): upserts every research row into the proper IQAC collection (journals → `Publication` type journal, patents → `Patent`, and so on), writes the generated ids back into the form rows, and sets `monthly_saved_at`. It also removes the same entry from *other* co-authors' drafts and emits `cross_faculty_update` so they are told.
- **Submit** (`POST /apar/mongo/submit`) is **refused unless `monthly_saved_at` is set**, which forces IQAC to be updated first.

8.5 IQAC entry with faculty approval

1. Head/HOD submits an entry that names faculty. `POST /iqac-approvals` (multipart): attachments are uploaded, faculty ids are extracted from the payload (`faculty_id`, `supervisor_id`, `committee_members[].member_id`...), each is checked to exist in `faculty`.
2. An `IqacApproval` document is created with one `pending` row per faculty member; each faculty member gets a notification (type `IQAC_APPROVAL_REQUEST`) over REST + socket.
3. Faculty open `/apar/iqac-approvals` (`GET /iqac-approvals/mine`) and call `POST /iqac-approvals/:id/decision` with approve or reject (an approver may attach corrected data).
4. **Any rejection**: creator is notified and the request is deleted permanently.
5. **All approved**: `finalizeApproval` looks the resource up in `iqacApprovalRegistry`, calls its `set()` (so flow 8.3 then runs as normal), stores `permanent_record_id` and notifies the creator. If that call fails the request is marked `failed`.

8.6 APAR lifecycle (status machine)

Draft → submit → Submitted → Reporting Officer → Forwarded by Reporting officer → Reviewing Officer → Accepted by Reviewing officer

Query Raised by Reporting officer or Query Raised by Reviewing officer → goes back to faculty, who edits and re-submits → Submitted

1. **Fill.** `GET /apar/mongo/form` loads or creates the Draft for (faculty, AY), runs the IQAC sync in memory on every load (persisting it only while the form is still a draft), refreshes Part I from the faculty record, and returns it. The faculty member works through Parts I–IV, saving drafts.
2. **Submit.** `submitForm` resolves the officers: the user's own `reporting_officer_id` / `reviewing_officer_id`, else someone with that apar_role in the same department, else anyone with that role (for reviewing, finally a Dean). If none found it errors "Contact office to add receiving and reporting officer". It sets `status = Submitted`, `timeline.submitted_at`, pushes a `history` entry, and mirrors the data to the Faculty record.
3. **Reporting.** The Reporting Officer opens `/apar/reporting` (`GET /reporting/pending` returns forms for officers assigned to them), fills Part V and calls `POST /reporting/submit` with status `Forwarded by Reporting officer` or `Query Raised by Reporting officer`. A query clears the timeline stamps and stores the comment; the faculty member is notified.
4. **Reviewing.** The Reviewing Officer's queue (`GET /reviewing/pending`) shows only forms with status `Forwarded by Reporting officer` (with `?archive=true` it shows every non-draft form); they fill Part VI and calls `POST /reviewing/submit` with `Accepted by Reviewing officer` or a query. A notification goes back to the faculty member.
5. **Monitor.** Dean dashboard (`GET /dean/status`) and IQAC dashboard (`aggregateAparMetrics`) show status counts. `GET /history` and `/dean/history` list past years.

Every transition appends to `history[]` (action, by, date, comment), which the timeline component displays.

8.7 File uploads: four different paths

Use	Route	Path of the bytes
IQAC attachments (paper PDF, certificates)	`POST /{resource}/set` with `upload.single`, or `POST /upload`	browser → multer to `public/temp` → **Cloudinary** (temp file deleted); the returned URL is stored on the record. `/upload` falls back to `public/uploads` and returns `/uploads/<file>` if Cloudinary env vars are missing.
APAR proof PDFs (Parts II, III)	`POST /apar/mongo/documents/temp` then Save / Save-as-Monthly	browser → `storage/temp/apar-documents/<uuid>.pdf` with an owner-checked sidecar JSON → the form stores the temp id → on save, `resolveAparDocuments` uploads to **MinIO** at `document/<name>/<ay>/<uuid>.pdf`, stores that object path in the form and deletes the temp. Failures roll back uploaded objects.

Optional / profile / avatar	`/documents/direct`, `/apar/auth/profile/avatar`, profile endpoints	browser → multer → **MinIO** directly (`optionaldocuments/…`, `profilepicture/…`, `profile/…`).
Reading a MinIO file	`GET /apar/mongo/document?path=…`	Checks the path starts with an APAR prefix (`document/`, `profile/`, ...), then streams it from MinIO. See the note in section 10.

8.8 Real-time channel (Socket.IO)

Room	Joined by	Events delivered
`user-<userId>`	every logged-in client (`join_notifications`)	`notification`
`apar-<facultyId>-<ay>`	AparForm / `useAparRealTimeSync`	`new_entry`, `update_entry`, `delete_entry`, `bulk_entries`, `apar_data_updated`, `cross_faculty_update`
`iqac-admin-<ay>`	IQAC monitors (optional)	`apar_update`
`IQAC_HEAD`, `HOD`	by role on `join_notifications`	role-wide broadcasts

Notifications are **persisted first** (`Notification` collection) and then pushed; `NotificationBell` loads history over REST and appends live ones, so nothing is lost if the socket is down.

8.9 Dashboard and reports

- **Dashboard**: `Dashboard.jsx` → `GET /dashboard/stats?academic_year&department_id`. `dashboard.controller.js` runs parallel aggregations over every model (counts by type, per-department splits, APAR status counts) and returns chart-ready data with colours. HODs are locked to their own department.
- **Reports**: `IqacReports.jsx` fetches every resource through its service, filters by the shared year/department filter (`iqacScopeFilter.js`) and builds PDF (jsPDF + autotable) or Excel (xlsx) in the browser. No server-side rendering.

8.10 Feedback analysis (AI)

Two implementations exist side by side:

- **Current UI path** (`FeedbackAnalysis.jsx`): parse the spreadsheet in the browser, build a prompt from `feedbackContexts.js`, call the Hugging Face router directly from the browser, render Markdown.
- **Backend path** (`POST /feedback/analyze/<type>`, `feedback.service.js`): multer memory upload, `xlsx` parse, Gemini (`GEMINI_API_KEY`), returns Markdown. Restricted to IQAC Head / HOD.

8.11 Faculty profile

`/apar/profile/self` (GET/PUT) lets a faculty member maintain a profile (basic info, qualifications, certificates, photo) stored in `faculty_profiles`; IQAC/Dean can read all profiles. When a Draft APAR is loaded, its Part I fields (name, email, designation, date of birth, phone, joining date) are refreshed from the faculty record so they never go stale.

9. Root-level and miscellaneous files

File	What it is
`README.md`	Title only (30 bytes, UTF-16).
`setup.md`	How to run with Docker; service URLs; seeding command.
`deploy.md`	Railway deployment: one service each for backend, frontend, MongoDB, MinIO; env var changes.
`SECURITY_AUDIT_REPORT.md`	Written security audit: vulnerabilities found, fixes (CSRF, rate-limit, helmet, cookie sessions, lockout, password policy, route guards).
`apar_presentation_content.md`	Slide-by-slide content for a project presentation.
`docker-compose.yml`	Local stack (section 2).
`requirements.txt`	A flat package list, documentation only; the real manifests are the two `package.json` files.

`rewrite_iqac_exports.js`	One-time script that rewrote export helpers in `IqacReports.jsx`. Not part of the app.
`diff.txt`	A saved git diff of `tableConfig.js`, UTF-16 encoded. Not used.
`frontend/update_profile.py`	One-time Python patcher for `Profile.jsx` validation strings (Windows path inside).
`backend/check_apar_*.js`, `test_db_*.js`, `inspect_db.js`, `insert_pawan_apar.js`, `update_pawan_profile.js`, `export_sample_form.js`	Developer debugging scripts that connect directly to MongoDB to inspect or patch specific records.
`backend/route-test-report.*`	Output of `tools/route_tester.js`.
`backend/dummy_*.json`	Sample users, faculty, students, departments.
`frontend/.vite/`	Vite dependency cache that was committed by accident.
`backend/public/uploads/*.pdf, *.jpg`	Uploaded files that were committed (local-storage fallback output).

Environment variables

Variable	Used for
`PORT`, `ORIGIN`	Backend port; comma-separated list of allowed CORS / Socket origins.
`MONGODB_URI`	Base URI; the code appends `/apar`.
`JWT_SECRET`, `JWT_EXPIRES_IN`, `AUTH_COOKIE_NAME`, `SESSION_TTL_MS`	Token signing and lifetime; session expiry stored on the user (default 12 h).
`CSRF_SECRET`	CSRF token signing (falls back to `JWT_SECRET`).
`RATE_LIMIT_MAX`, `AUTH_RATE_LIMIT_MAX`, `MAX_FAILED_LOGIN_ATTEMPTS`, `LOGIN_LOCKOUT_MS`, `MIN_PASSWORD_LENGTH`	Security tuning.
`MINIO_ENDPOINT / PORT / USE_SSL / BUCKET / ROOT_USER / ROOT_PASSWORD`	MinIO connection.
`CLOUDINARY_*`	IQAC attachment storage.
`GEMINI_API_KEY`	Backend feedback analysis.
`ADMIN_PROVISION_TOKEN`, `ALLOW_AUTO_PROVISION`, `DEFAULT_INITIAL_PASSWORD`	User provisioning controls (auto-provision is ignored in production).
`VITE_BASEURL`, `VITE_HUGGINGFACE_API_KEY`, `VITE_HF_*`	Frontend: API root and AI settings (baked into the JS bundle at build time).

10. Things worth knowing

These came up while reading. None were changed. They are listed as a map of where the sharp edges are.

Security-relevant

- **The Hugging Face key is exposed to every browser.** `VITE_HUGGINGFACE_API_KEY` is compiled into the frontend bundle, and `huggingface.service.js` calls the provider directly. The server-side Gemini route exists and would keep the key private.
- **APAR documents are readable by any logged-in user who knows the path.** `GET /apar/mongo/document?path=…` (`streamDocument`) only checks that the path starts with an allowed prefix; unlike the other APAR endpoints it does not call `assertFacultyAccess`. Paths contain the faculty name, academic year and a UUID, so they are hard but not impossible to guess.
- **`POST /auth/register` is unauthenticated** and relies only on a shared `x-provision-token` header matching `ADMIN_PROVISION_TOKEN`; if that env var is unset, it refuses everyone.
- **Hard-coded defaults:** MinIO falls back to `minioadmin / minioadmin` and `setup.md` lists mongo-express as `admin / password123`. Fine locally, wrong in a deployment.
- **`/api/v1/upload` is mounted before `csrfProtection`** in `app.js`, so uploads through that route skip the CSRF check (it still requires login).
- `must-change-password.middleware.js` is never mounted, so the "must change password" flag is stored but not enforced by the server.

Behaviour to be aware of

- **One session per user across both portals.** IQAC and APAR logins both write `users.session_token`; signing in to one portal logs the same user out of the other.
- **Two copies of research data** (IQAC collections vs `aparforms.research`) kept in sync by code, not by the database. Removing a row in an APAR draft deletes the IQAC record; the sync matches academic years by string variants (`2026-27` vs `2026-2027`), which is why the global normaliser exists.
- **The auto-sync trigger ignores the academic year it is given.** Data-access computes an AY from `year_of_publication`, but `triggerAparAutoSync` deliberately syncs *every* draft form of that faculty member; each form's own `ay` then decides which IQAC records it pulls in.
- **Duplicate policy:** journal `set()` merges silently into an existing identical paper (adds authors) instead of failing.
- The APAR submit path has its notification calls commented out; notifications are sent for IQAC sync, approvals, reporting and reviewing, but not on faculty submit.
- **Ids are strings, and several are ambiguous.** A request may carry a Mongo ObjectId, a `user_id` or a `faculty_id`; `resolveToReadableId` is called in many places to reconcile them.

Cleanup candidates

- **Dead commented-out copies:** `AddPage.jsx` (~780 lines), `Dashboard.jsx` (~550 lines) and `Landing.jsx` each start with a fully commented-out older version before the live component.
- **Near-identical files ×37:** routes, controllers, data-access and services repeat the same CRUD skeleton per resource. A small factory would remove thousands of lines, but the per-resource merge/transform logic would have to be preserved.
- **Giant files:** `apar.mongo.controller.js` (2.6k lines), `AparForm.jsx` (2.7k), `tableConfig.js` (2.1k), `Dashboard.jsx` (1.5k).
- **Tracked artefacts:** `frontend/.vite/`, `backend/public/uploads/*`, `diff.txt`, `route-test-report.*`, `backend/dummy_*.json`, and the many `check_apar_*.js / test_*.js` scripts in `backend/` root.
- **Duplicate declarations:** `app.set('trust proxy', 1)` twice in `app.js`; `duplicateDetectionConfig.js` and `_generated.js`; two role files per side (IQAC vs APAR).
- **Unused:** `parseFormData.middleware.js` (empty), `must-change-password.middleware.js` (not mounted), `puppeteer` in frontend dependencies (no import found in `src/`), `axios-cookiejar-support` and `tough-cookie` in backend dependencies (used only by the tester, if at all).

End of document.